

- Problem 1 Consider the following bank database with 6 tables. For each table, the primary key is underlined.

| Table            | Attributes                                                         |
|------------------|--------------------------------------------------------------------|
| <i>branch</i>    | ( <u>branch_name</u> , <u>branch_city</u> , <u>assets</u> )        |
| <i>customer</i>  | ( <u>ID</u> , <u>customer_name</u> , <u>street</u> , <u>city</u> ) |
| <i>loan</i>      | ( <u>loan_number</u> , <u>branch_name</u> , <u>amount</u> )        |
| <i>borrower</i>  | ( <u>CustomerID</u> , <u>loan_number</u> )                         |
| <i>account</i>   | ( <u>account_number</u> , <u>branch_name</u> , <u>balance</u> )    |
| <i>depositor</i> | ( <u>CustomerID</u> , <u>account_number</u> )                      |

1. Find the ID and name of customers who live in "Boston".

$$\Pi_{ID, customer\_name}(\sigma_{city="Boston"}(customer))$$

2. Find the ID of each borrower who has a loan in branch "Edison."

$$\Pi_{CustomerID}(\sigma_{branch\_name="Edison"}(borrower \bowtie_{borrower.loan\_number=loan.loan\_number} loan))$$

3. Find the ID and names of customers who have deposits in "Edison" branch but do not have loans in "Edison" branch.

$$\begin{aligned} &\Pi_{ID, customer\_name}(\sigma_{branch\_name="Edison"}( \\ &customer \bowtie_{customer.ID=depositor.CustomerID} \\ &depositor \bowtie_{depositor.account\_number=account.account\_number} account)) \\ &- \Pi_{ID, customer\_name}(\sigma_{branch\_name="Edison"}( \\ &customer \bowtie_{customer.ID=borrower.CustomerID} borrower \bowtie_{borrower.loan\_number=loan.loan\_number} loan)) \end{aligned}$$

4. Find the ID and names of customers who have at least one account in every branch with assets exceeding 1,000,000.

$$\text{Let } milAcc = \Pi_{ID, customer\_name, branch\_name}$$

$$(\sigma_{balance > 1,000,000}(customer \bowtie_{customer.ID=account.customer\_ID} account)) \text{ and}$$

$$B = \Pi_{branch\_name}(branch). \text{ Then the desired result can be found via}$$

$$\Pi_{ID, customer\_name}(milAcc) - \Pi_{ID, customer\_name}(\Pi_{ID, customer\_name}(milAcc) \times B - milAcc)$$

Or the division operator between *milAcc* and *B*, via finding all possible combinations between branch and accounts with at least a million dollars, excluding the ones that actually exists, then excluding the set of non-sense names (or those who didn't already have entries corresponding to the cartesian product), from the set of total accounts.

- Problem 2 Consider the following employee database with 3 tables. For each table, the primary key is underlined.

| Table           | Attributes                                                       |
|-----------------|------------------------------------------------------------------|
| <i>employee</i> | ( <u>ID</u> , <u>person_name</u> , <u>street</u> , <u>city</u> ) |
| <i>works</i>    | ( <u>EmployeeID</u> , <u>company_name</u> , <u>salary</u> )      |
| <i>company</i>  | ( <u>company_name</u> , <u>city</u> )                            |

1. Please show the relational algebra for the following queries.

(a) Find ID and names of employees working for NVIDIA.

$\Pi_{ID, person\_name}(\sigma_{company\_name = "NVIDIA"}(employee \bowtie_{employee.ID = works.EmployeeID} works))$

(b) Find ID and names of employees who work and live in different cities.

$\Pi_{ID, person\_name}(\sigma_{employee.city \neq company.city}(employee \bowtie_{employee.ID = works.EmployeeID} works \bowtie_{works.company\_name = company.company\_name} company))$

2. Please draw a query tree for the query (2). Find ID and names of employees who work and live in different cities.

